Natural Language Processing with Generative AI

Course Summary

Course Overview

This course explores the foundations and advancements of Natural Language Processing (NLP) with a focus on Generative AI applications. It covers the evolution of NLP from traditional techniques to transformer-based architectures and large language models (LLMs), emphasizing their ability to understand, process, and generate human language. Learners gain hands-on experience with key concepts such as word embeddings, transformers, prompt engineering, and Retrieval-Augmented Generation (RAG), enabling them to design reliable and context-aware AI systems.

Week 0: Introduction to Natural Language Overview

Overview of NLP

Natural Language Processing (NLP) is a field of artificial intelligence that focuses on enabling machines to understand, interpret, and generate human language. It bridges computer science, linguistics, and machine learning, allowing systems to process text and speech data meaningfully. From basic tasks like text classification to advanced applications like conversational agents, NLP provides the foundation for many modern GenAI solutions.

History of NLP

The development of NLP has evolved from rule-based systems in the 1950s and 1960s, where handcrafted grammar rules dominated, to the statistical models of the 1980s and 1990s, which introduced probabilistic methods for language processing. The 2000s saw the rise of machine learning techniques, followed by the deep learning revolution in the 2010s, which significantly improved tasks like machine translation, sentiment analysis, and question answering. Today, large language models (LLMs) drive the GenAI era, offering human-like fluency and reasoning capabilities.

Applications of NLP

NLP powers a wide range of applications across industries. In business, it is used for chatbots, customer support automation, and sentiment analysis. In healthcare, it helps analyze clinical notes and assist in medical decision-making. Search engines, virtual assistants, recommendation systems, and generative AI tools like text summarizers or translators all rely on NLP. Its ability to process unstructured text and speech data makes it a cornerstone technology for modern AI-driven systems.

Sequential and Non-Sequential Processing

Language data can be processed in two major ways: sequentially or non-sequentially. Sequential methods handle text step by step, preserving order but struggling with long-term dependencies, while non-sequential methods process entire sequences in parallel, enabling better context capture and scalability.

Aspect	Sequential Processing	Non-Sequential Processing
Processing Style	Step-by-step (word by word)	Parallel (entire sequence at once)
Context Handling	Good for short-term dependencies, weak for long-term	Excellent at capturing long-range dependencies
Training Speed	Slower due to the sequential nature	Faster due to parallelisation
Scalability	Limited scalability to very long sequences	Highly scalable, suited for large datasets
Model Examples	RNN, LSTM, GRU	Transformer, BERT, GPT, T5

GenAI Relevance	Early NLP era, less common today	Backbone of modern LLMs and GenAI applications

Week 1: Word Embeddings

Introduction to Word Embeddings

Word embeddings are dense vector representations of words that capture their meanings, relationships, and contexts in a numerical form understandable by machines. Unlike traditional one-hot encodings, which are sparse and lack semantic information, embeddings place similar words closer together in vector space. This representation enables models to generalize better and perform complex language tasks effectively.

Word2Vec

Word2Vec, developed by Google in 2013, is one of the most influential models for learning word embeddings. It is a shallow, two-layer neural network that represents words in a continuous vector space based on context. The key idea is that words used in similar contexts have similar meanings.

Example:

- Consider the analogy: "*king–man + woman ≈ queen.*"
  Word2Vec captures this relationship because the vector difference between *king* and *man* is similar to the difference between *queen* and *woman*.

- Similarly, words like *Paris* and *France* will be close in the embedding space, just as *Tokyo* and *Japan* are.

This ability to encode semantic and syntactic relationships revolutionized NLP and became the foundation for later models.

CBOW and Skip-gram

Word2Vec uses two training architectures: **CBOW (Continuous Bag of Words)** and **Skip-gram**.

- CBOW predicts a target word from its surrounding context words.
- Skip-gram does the opposite: it predicts surrounding context words given a target word.

Detailed Comparison: CBOW vs Skip-gram

Aspect	CBOW (Continuous Bag of Words)	Skip-gram
Objective	Predict the target word from multiple context words	Predict surrounding context words from the target word
Direction of Training	Context ->Target	Target ->Context
Input	Embeddings of context words	Embedding of a single target word
Output	Single target word	Multiple context words
Efficiency	More efficient, faster to train, good for frequent words	Computationally heavier, but captures richer information
Performance with Rare Words	Struggles with rare/infrequent words	Performs well with rare and domain-specific words
Context Window Example	Sentence: "*The cat sat on the mat*", window=2 -> Inputs: [The, cat, on, the] -> Predict *sat*	Sentence: "*The cat sat on the mat*", window=2 -> Input: *sat* -> Predict [The, cat, on, the]
Strengths	Captures general semantic meaning, faster on large datasets	Better semantic precision, especially in low-resource or domain-specific vocabularies

Weaknesses	Loses detail for rare words; averaging context may blur nuances	Slower training due to multiple predictions per word
Use Cases	Large corpora with frequent vocabulary (e.g., news articles)	Smaller datasets, tasks requiring rare/technical vocabulary (e.g., scientific text)

GloVe

GloVe (Global Vectors for Word Representation), developed by Stanford, combines the strengths of global matrix factorization (like LSA) and local context window methods (like Word2Vec). It constructs embeddings by analyzing global word co-occurrence statistics across a corpus. Unlike Word2Vec, which is predictive, GloVe is count-based, making it especially effective at capturing global semantic relationships.

Applications of Word Embeddings

Word embeddings are a core building block in modern NLP because they convert text into meaningful numerical representations that preserve semantic relationships. These embeddings enable models to generalize across tasks and reduce the complexity of handling raw text data.

Key Applications:

1. **Text Classification** – Used in spam detection, sentiment analysis, and topic categorization by providing dense vector inputs to classifiers.
2. **Machine Translation** – Embeddings capture semantic relationships across languages, improving translation accuracy.
3. **Information Retrieval & Search Engines** – Word embeddings enhance query–document matching by understanding semantic similarity (e.g., “car” ≈ “automobile”).
4. **Question Answering & Chatbots** – They help systems interpret user queries and generate relevant, context-aware responses.

5. **Recommendation Systems** – Embeddings represent user preferences and item descriptions, allowing better personalization.
6. **Named Entity Recognition (NER)** – Useful in tagging entities like names, places, or organizations in unstructured text.
7. **Generative AI Models** – Embeddings are the foundation for LLMs, enabling tasks like summarization, paraphrasing, and creative text generation.

Key Functions:

- **Word2vec:** A technique to convert words into numerical vectors such that words with similar meaning have closer vector representations.
- **PorterStemmer()**: A function from the NLTK library used to reduce words to their root or stem form by removing common suffixes.
- **glove2word2vec()**: Converts GloVe word embeddings into Word2Vec format for compatibility with Word2Vec-based tools.
- **train_test_split(X, y, test_size, random_state)**: Splits the data into training and testing sets.

Week 2: Transformers

Introduction to Transformers

Transformers, introduced in the paper *"Attention is All You Need"* (2017), revolutionized NLP by replacing recurrent and convolutional models with a parallelized architecture based on attention. Unlike RNNs or LSTMs, transformers process sequences in parallel, making them much faster and better at handling long-range dependencies. This design paved the way for large-scale language models like BERT, GPT, and T5, which power today's GenAI applications.

Introduction to Attention Mechanism

The attention mechanism allows models to focus on the most relevant parts of the input when making predictions. Instead of treating all words equally, attention assigns weights that determine how much each word contributes to understanding a target word.

Example:
Consider the sentence: "*The cat chased the mouse because it was hungry.*"

- A simple model may struggle to know whether "*it*" refers to a *cat* or a *mouse*.
- With **self-attention**, the model assigns a higher weight between "*it*" and "*cat*", correctly capturing the dependency.

This ability to dynamically focus on relevant words enables transformers to capture context more effectively than traditional RNNs or CNNs.

Components of a Transformer

Transformers are built from repeated blocks containing several key components. Each plays a role in enabling efficient parallel processing, context capture, and stable training.

1. **Input Embeddings & Positional Encoding**
    - Words are first converted into embeddings (dense vectors).
    - Since transformers lack recurrence, positional encoding is added to preserve word order in the sequence.
2. **Self-Attention**
    - Allows each word to attend to all other words in a sequence.
    - Captures dependencies regardless of distance (e.g., "*The cat … it*").
    - Works using Query, Key, and Value matrices to compute relevance scores (attention weights).
3. **Multi-Head Attention**
    - Instead of one attention mechanism, the model uses multiple attention "heads."
    - Each head learns different types of relationships (e.g., syntactic vs. semantic).

- Outputs are concatenated to form a richer representation.

4. **Masking**
    - Masking in Transformers blocks irrelevant or future tokens during training so the model only learns from valid context.
5. **Feed-Forward Neural Network (FFN)**
    - After attention, a position-wise feed-forward network transforms the representation.
    - Adds non-linearity and improves learning capacity.
6. **Skip (Residual) Connections**
    - Add the input of a layer directly to its output (before normalization).
    - Prevent vanishing gradients and allow deeper networks to train more effectively.
7. **Layer Normalization**
    - Normalizes the output of each sub-layer for stable training and faster convergence.
8. **Encoder and Decoder Blocks**
    - **Encoder:** Stacks multiple layers of self-attention + FFN to build input representations.
    - **Decoder:** Uses masked self-attention + cross-attention (attending to encoder output) to generate sequences step by step.

Types of Transformer Architectures

1. Encoder-Only Models

- **Structure:** Use only the encoder stack of the transformer.
- **Working:** These models focus on understanding and representing input text by capturing deep contextual embeddings.

- **Strengths:** Excellent for classification, sentence similarity, and tasks where no text generation is required.
- **Example:**
    - **BERT (Bidirectional Encoder Representations from Transformers)** – Learns bidirectional context for tasks like sentiment analysis and question answering.

2. Decoder-Only Models

- **Structure:** Use only the decoder stack of the transformer.
- **Working:** Generate output word by word in an autoregressive manner, predicting the next token based on previous ones.
- **Strengths:** Highly effective for text generation, completion, and dialogue systems.
- **Examples:**
    - **GPT family (GPT-2, GPT-3, GPT-4), LLaMA, Falcon** – State-of-the-art generative models.
    - Used in chatbots, story generation, and code generation (e.g., Codex).

3. Encoder–Decoder Models

- **Structure:** Combine an encoder to process input text and a decoder to generate output sequences.
- **Working:** Input is encoded into contextual embeddings by the encoder, and the decoder generates a target sequence conditioned on this representation.
- **Strengths:** Best for sequence-to-sequence tasks where both input and output are text.
- **Examples:**
    - **T5 (Text-to-Text Transfer Transformer)** – Treats all NLP tasks as text-to-text.
    - **BART, mBART, MarianMT, Pegasus** – Used for translation, summarization, and text rewriting.

Applications of Transformers

1. **Machine Translation**
    - Transformer-based models (e.g., MarianMT, mBART) are widely used for translating between languages with high accuracy.
    - Example: English ->German translation using encoder–decoder models.
2. **Text Summarization**
    - Abstractive summarization models like BART, T5, and Pegasus generate human-like summaries of long documents.
    - Used in news aggregation, research paper summarization, and business reporting.
3. **Question Answering & Information Retrieval**
    - BERT-based models excel at extractive QA (finding answers in passages).
    - GPT-based models perform generative QA by constructing natural language responses.
4. **Conversational AI & Chatbots**
    - Decoder-only models like GPT-3, GPT-4, and LLaMA power intelligent assistants and customer support bots.
    - Provide context-aware, coherent multi-turn dialogues.
5. **Text Classification**
    - Encoder-only transformers (like BERT, RoBERTa) are used for sentiment analysis, spam detection, and topic categorization.
6. **Code Generation and Assistance**
    - Models like Codex and Code LLaMA are trained on programming languages to generate, debug, and explain code.
7. **Multimodal Applications**
    - Transformers extend beyond text:

- **Vision Transformers (ViT):** For image recognition.
- **CLIP:** Aligns text and images (used in image search and captioning).
- **Multimodal GPTs:** Handle text, image, and even audio inputs together.

Key functions:

- pipeline("sentiment-analysis"): Creates a pre-trained Hugging Face pipeline to classify text sentiment (positive, negative, or neutral).
- train_test_split(X, y, test_size, random_state): Splits the data into training and testing sets.
- np.dot(embedding_matrix, query_embedding): Computes the dot product between embedding_matrix and query_embedding, often used to measure similarity or perform vector projections.
- T5Tokenizer.from_pretrained("google/flan-t5-large"): Loads the pre-trained T5 tokenizer for the flan-t5-large model from Hugging Face, enabling text tokenization and detokenization for that model.
- T5ForConditionalGeneration.from_pretrained("google/flan-t5-large", load_in_8bit=True, device_map="auto"): Loads the pre-trained FLAN-T5 large model for conditional text generation in 8-bit precision and automatically maps it across available devices (e.g., GPU/CPU).

Week 3: Large Language Models and Prompt Engineering

Large Language Models (LLMs)

Large Language Models are advanced AI systems trained on billions of words, capable of understanding, generating, and reasoning with human language. Built primarily on the transformer architecture, LLMs like GPT, LLaMA, and Falcon excel at tasks such as text generation, summarization, translation, and dialogue. Their scale (number of parameters and data size) allows them to capture nuanced patterns of language, making them foundational in Generative AI.

Training of LLMs

LLMs are trained in two major stages:

1. **Pre-training:**
    - Models are trained on massive text corpora using objectives like causal language modelling (CLM) or masked language modelling (MLM).
    - This helps them learn grammar, facts, and reasoning abilities.
2. **Fine-tuning:**
    - LLMs are adapted to specific tasks or domains using smaller labelled datasets.
    - Techniques include supervised fine-tuning (SFT), reinforcement learning from human feedback (RLHF), and parameter-efficient tuning (LoRA, adapters).
    - Example: ChatGPT uses RLHF to align responses with human preferences.

Evaluation of LLMs

Evaluating LLMs involves both **quantitative** and **qualitative** methods:

- **Quantitative Metrics:** Perplexity, BLEU, ROUGE, accuracy, and F1-score for downstream tasks.
- **Qualitative Evaluation:** Human judgement of fluency, coherence, factual accuracy, and safety.
- **Robustness Testing:** Checking for bias, hallucinations, and adversarial prompts.
- **Benchmark Suites:** GLUE, SuperGLUE, MMLU for NLP tasks; HELM and BIG-bench for LLM evaluation.

Applications of LLMs

Large Language Models have diverse applications across industries, ranging from conversational AI to domain-specific expertise. The table below highlights key applications, example models, and their real-world use cases.

Application	Example Models	Real-world Use Cases
Conversational AI & Virtual Assistants	ChatGPT, Google Bard, LLaMA	Customer support chatbots, enterprise assistants, personal productivity helpers
Content Generation & Creativity	GPT-4, Jasper, Claude	Blog writing, marketing copy, story/poetry generation
Summarization & Paraphrasing	BART, Pegasus, GPT-4	News aggregation, academic paper summaries, and meeting note generation
Machine Translation	mBART, MarianMT, GPT-based translators	Multilingual communication, global business operations
Information Retrieval & Question Answering	BERT, GPT-4, RAG-based models	Legal research, healthcare Q&A, financial analysis assistants
Programming Assistance	Codex, Code LLaMA	Code generation, debugging, developer copilots
Education & Personalized Learning	GPT-4, Claude, domain-tuned LLMs	Interactive tutors, quiz generation, concept explanations
Multimodal Applications	GPT-4 (Vision), CLIP, Flamingo	Text–image Q&A, visual reasoning, caption generation
Domain-Specific AI	BioBERT, LegalBERT, FinBERT	Biomedical research, legal document analysis, and financial reporting

Prompt Engineering

Prompt engineering is the process of crafting effective inputs (prompts) to guide Large Language Models (LLMs) in generating accurate and useful outputs. Since the response of an LLM depends heavily on how the question or task is framed, well-designed prompts provide clarity, context, and constraints.

For example, instead of asking "Explain transformers", a better prompt would be "Explain transformers in NLP in simple terms with an example." Techniques like giving examples (few-shot prompting), step-by-step instructions (chain-of-thought), or assigning roles ("You are a tutor…") help improve results.

In short, prompt engineering is an essential skill for using LLMs effectively in real-world applications.

Strategies for Devising Effective Prompts

Designing effective prompts is key to improving the quality of responses from Large Language Models. Below are some widely used strategies:

1. **Instruction-based Prompts**
    - Clearly state what the model should do.
    - Example: *"Classify the following review as positive, negative, or neutral: 'The product was amazing, but delivery was late.'"*
    - Works well for tasks like classification, summarization, or translation.
2. **Zero-shot and Few-shot Prompting**
    - **Zero-shot:** Ask the model to perform a task without examples.
      "Translate this sentence into French: 'I love learning NLP.'"
    - **Few-shot:** Provide a few examples to guide the model's response style.
      "Review: 'Great service.' ->Positive. Review: 'Food was cold.' ->Negative. Review: 'Ambience was nice.' ->Positive. Now classify: 'The staff was rude.'"
3. **Chain-of-Thought Prompting**
    - Encourage step-by-step reasoning for complex tasks.

- Example: *"Solve the math problem step by step: A train travels 60 km in 1.5 hours. What is its speed?"*
- Useful in reasoning-heavy tasks like math, logic puzzles, or decision-making.

4. **Role-based Prompting**
   - Assign a role to shape the model's response style.
   - Example: *"You are a data analyst. Summarize the key trends in this dataset for a business manager."*
   - Helps in generating domain-specific and professional outputs.
5. **Contextual Expansion**
   - Provide background or additional details to reduce ambiguity.
   - Example: Instead of *"Summarize this text"*, use *"Summarize this article for high school students in 3 bullet points."*
   - Ensures more tailored and audience-specific outputs.

Ethical Usage of AI

- AI tools should be used responsibly to prevent harm and misuse.
- Always review outputs, cite sources, and avoid spreading misinformation.
- Both developers and users share responsibility for safe and fair usage.
- The focus is on ensuring AI creates positive impact with accountability.

Key functions:

- **hf_hub_download():** Hugging Face utility function that downloads a specific file from a model, dataset, or Space repository on the Hugging Face Hub to your local cache and returns its local path.

Week 4: Retrieval Augmented Generation

Introduction to Retrieval-Augmented Generation (RAG)

What is RAG?

Retrieval-Augmented Generation (RAG) is an advanced framework that improves Large Language Models (LLMs) by combining retrieval (fetching information from external sources) with generation (producing natural language answers). Instead of relying only on what the model learned during training, RAG allows the system to pull in up-to-date and domain-specific knowledge when answering.

Why do we need RAG?

- LLMs are trained on fixed datasets and may not know the latest information.
- They often hallucinate, producing factually incorrect but fluent responses.
- Many applications require grounded answers (e.g., legal advice, healthcare Q&A, enterprise knowledge).

Key Motivation:

- RAG reduces hallucination by anchoring answers in retrieved, real-world data.
- It makes LLMs scalable and adaptable across different domains without retraining.
- This ensures accuracy, trustworthiness, and explainability in AI-powered systems.

Example:

User asks: "*What are the side effects of drug X?*"

- A standard LLM may generate a generic or incorrect response.
- With RAG, the system retrieves information from medical research papers or drug databases, then generates a precise and trustworthy answer.

Working Structure of RAG

RAG works as a step-by-step pipeline:

1. **User Query** – The user asks a question.

2. **Embedding** – The query is converted into a numerical vector.
3. **Retrieval** – Relevant documents are fetched from a knowledge base.
4. **Augmentation** – The retrieved context is added to the user query.
5. **Generation** – The LLM uses both query + retrieved data to create an accurate response.

Building Blocks of RAG

- **Retrieval Module:**
    - Uses embeddings and semantic search to find the most relevant documents.
    - Ensures that the LLM is provided with correct and contextually useful data.
- **Generation Module:**
    - Takes the retrieved passages + query and generates a natural, human-like response.
    - Ensures that the answer is coherent and easy to understand.
- **Integration:**
    - Retrieval ensures **accuracy**, generation ensures **fluency**.
    - Together, they create a balance between factual correctness and natural language.

Vector Database and Semantic Search

- **Vector Database**
    - Stores documents in the form of **embeddings** (dense numerical vectors) that capture semantic meaning rather than just keywords.
    - This makes it possible to retrieve relevant passages even when wording differs.
    - Examples: **FAISS**, **Pinecone**, **Weaviate**, **Milvus**.
- **Semantic Search**
    - Goes beyond keyword search by focusing on **meaning**.

- Example: A query like *"symptoms of heart attack"* can also match documents describing *"signs of myocardial infarction."*
- Greatly improves retrieval quality, especially in specialized fields like medicine, law, or research.

Devising and Evaluating Prompts for RAG

- **Prompt Design in RAG**
  - Prompts should instruct the model to use the retrieved context rather than rely solely on prior knowledge.
  - Example: *"Using only the information from the retrieved documents, answer the following question..."*
  - Helps reduce hallucination and ensures the response is factually grounded.
- **Evaluation of RAG Systems**
  - *Relevance*: Retrieved documents must closely match the query.
  - *Faithfulness*: The generated output should be consistent with retrieved evidence, not invented.
  - *Usefulness*: The final response should be clear, complete, and actionable for the end-user.

Key functions:

- **PyPDFLoader():** A class from LangChain that loads and splits PDF files into individual pages as text documents for easy processing in LLM pipelines.
- **RecursiveCharacterTextSplitter():** A LangChain utility that splits long text into smaller chunks by recursively breaking at separators (like paragraphs, sentences, or words) while trying to keep chunks within a specified size.
- **SentenceTransformerEmbeddings():** A LangChain class (alias for HuggingFaceEmbeddings) that leverages SentenceTransformers models to convert text into dense vector embeddings for semantic search and related tasks.

- **Chroma.from_documents():** A LangChain class method that builds a Chroma vector store from a list of Document objects, optionally with embeddings, IDs, persistence settings, and other collection options.

- **vectorstore.as_retriever():** A LangChain method that wraps a vector store (e.g., Chroma, FAISS, Pinecone) into a retriever interface, enabling you to query the store for relevant documents by similarity, MMR, or threshold-based searches.